

第二课 深入学习solidity

目录

01

高阶语法学习

CALL、DELEGATECALL

02

理解发送交易过程

transfer、send

03

深入理解Create2
合约实操

04

多签钱包演示

abi.encodeXXX系列函数的区别

The background features a light green, textured rectangular area with a thin white border. This is set against a dark green, wood-grain-like background. The corners and edges are decorated with various green leaves and small orange flowers. A large teal circle is centered on the page.

01

高阶语法学习

call 和 delegatecall方法

教育教学

思考

call方法

特点是什么？

什么场景下用过Call呢？

delegatecall 方法

○ ○ ○

前置概念ABI

```
Contract ABI </> API
[{"constant":true,"inputs":[],"name":"name","outputs":[{"name":"","type":"string"}],"payable":false,"stateMutability":"view","type":"function"},
{"constant":false,"inputs":[{"name":"guy","type":"address"},{"name":"wad","type":"uint256"}],"name":"approve","outputs":
[{"name":"","type":"bool"}],"payable":false,"stateMutability":"nonpayable","type":"function"},{"constant":true,"inputs":[],"name":"totalSupply","outputs":
[{"name":"","type":"uint256"}],"payable":false,"stateMutability":"view","type":"function"},{"constant":false,"inputs":[{"name":"src","type":"address"},
{"name":"dst","type":"address"},{"name":"wad","type":"uint256"}],"name":"transferFrom","outputs":
[{"name":"","type":"bool"}],"payable":false,"stateMutability":"nonpayable","type":"function"},{"constant":false,"inputs":
[{"name":"wad","type":"uint256"}],"name":"withdraw","outputs":[],"payable":false,"stateMutability":"nonpayable","type":"function"},{"constant":true,"inputs":
[{"name":"decimals","outputs":[{"name":"","type":"uint8"}],"payable":false,"stateMutability":"view","type":"function"},{"constant":true,"inputs":
[{"name":"","type":"address"}],"name":"balanceOf","outputs":[{"name":"","type":"uint256"}],"payable":false,"stateMutability":"view","type":"function"},
{"constant":true,"inputs":[],"name":"isContract","outputs":[{"name":"","type":"bool"}],"payable":false,"stateMutability":"view","type":"function"}]
```

函数调用

```
address public constant calledaddress = 0x9d83e140330758a8fFD07F8Bd73e86ebcA8a5692;
function calling() public {
    (bool success, bytes memory result) =calledaddress.call(abi.encodeWithSignature("count()"));
}
```

存在函数

不存在

```
[vm] from: 0x5B3...eddC4 to: caller.caller_error() 0xc9...2b4bD value: 0 wei
transact to caller.caller_error errored: Error occurred: revert.
```


call特点

返回一个元组，包含两个值，一个是布尔值，表示交易成功或失败，另一个是字节(bytes)，其字节是执行函数的返回值经过了abi编码(如果执行函数无返回值则返回为空)

- ❁ 没有gas限制，可以支持对方合约fallback()或receive()函数实现复杂逻辑

转账失败，不会revert，所以通常需要与❁ revert 函数一同使用，revert会撤销自事务开始以来对区块链状态的所有更改

delegatecall方法

返回一个元组，包含两个值，一个是布尔值，表示交易成功或失败，另一个是字节(bytes)，其字节是执行函数的返回值经过了abi编码(如果执行函数无返回值则返回为空)

❁ 包括call方法

编码：data = abi.encode(value) [将 value类型的数据编码成bytes数据]

解码：data = abi.decode(data,(uint256)) 解码data中的uint256数据 [从bytes类型的数据中，提取出uint256类型的值]

本质区别

上下文不同，调用delegatefall方法尤其需要注意存储槽的问题

❁ 存储原理

- 智能合约的存储三个特点：**持久，可读，可写**。一旦上链，合约将永久存在，合约的存储是开发是预先分配的，读取数据时免费的，但是写入数据需要gas，因为你改变了合约的状态。
- 智能合约**只能读取自己上下文中的数据**，如果要读取其他智能合约的数据，需要由目标合约定义接口。或者使用方法，但也需要看目标合约是否限制了访问权限

delegate call storage

⤴ 向上拉以精确地调整播放进度

Storage	
	Storage region 1
	Storage region 2
	Storage region 3
	Storage region 4
	Storage region 5
	Storage region 6
	Storage region 7
	Storage region 8
	Storage region 9
	Storage region 10
	Storage region 11
	Storage region 12
	Storage region 13
	Storage region 14
	Storage region 15
	Storage region 16
	Storage region 17
	Storage region 18
	Storage region 19
	Storage region 20
	Storage region 21
	Storage region 22
	Storage region 23
	Storage region 24
	Storage region 25
	Storage region 26
	Storage region 27
	Storage region 28
	Storage region 29
	Storage region 30
	Storage region 31
	Storage region 32
	Storage region 33
	Storage region 34
	Storage region 35
	Storage region 36
	Storage region 37
	Storage region 38
	Storage region 39
	Storage region 40
	Storage region 41
	Storage region 42
	Storage region 43
	Storage region 44
	Storage region 45
	Storage region 46
	Storage region 47
	Storage region 48
	Storage region 49
	Storage region 50
	Storage region 51
	Storage region 52
	Storage region 53
	Storage region 54
	Storage region 55
	Storage region 56
	Storage region 57
	Storage region 58
	Storage region 59
	Storage region 60
	Storage region 61
	Storage region 62
	Storage region 63
	Storage region 64
	Storage region 65
	Storage region 66
	Storage region 67
	Storage region 68
	Storage region 69
	Storage region 70
	Storage region 71
	Storage region 72
	Storage region 73
	Storage region 74
	Storage region 75
	Storage region 76
	Storage region 77
	Storage region 78
	Storage region 79
	Storage region 80
	Storage region 81
	Storage region 82
	Storage region 83
	Storage region 84
	Storage region 85
	Storage region 86
	Storage region 87
	Storage region 88
	Storage region 89
	Storage region 90
	Storage region 91
	Storage region 92
	Storage region 93
	Storage region 94
	Storage region 95
	Storage region 96
	Storage region 97
	Storage region 98
	Storage region 99
	Storage region 100

0:00



不可变和常量变量

```
contract Caller {  
    uint256 private immutable a = 3;  
    function getValueDelegate(address called) public pure returns (uint256) {  
        (bool success, bytes memory data) = B.delegatecall(  
            abi.encodeWithSignature("getValue()"));  
        return abi.decode(data, (uint256)); // is this 3 or 2?  
    }  
}
```

```
contract Called {  
    uint256 private immutable a = 2;  
  
    function getValue() public pure returns (uint256) {  
        return a;  
    }  
}
```


特殊场景

不可变(immutable)或常量变量(constant)不是真正的状态变量：它们不占用槽

全局变量



1. msg.sender: 调用当前合约的账户地址
2. msg.value: 当前调用中附带的以太币数量
3. msg.data : 调用中携带的原始数据（输入数据）

如果使用了delegatecall方法，在called合约中使用了msg.sender之类的全局变量，其msg.sender对应的是caller合约的msg.sender，因为其上下文始终是caller合约，而msg.sender的根据就是根据上下文而定的。



02

理解交易发送过程

交易的过程

日常
交易

思考：有哪些业务或者场景会用到交易方法呢？它包含哪些方法呢？

小李偶然得到了一笔空投，大概1000u，碰巧遇到好伙伴小明需要800u参与launchpad活动，并说给他转账800u，下周还他1000u。那小李可以使用那种方式实现呢？

场景

交易
数据

你了解ETH交易结构么？组成都有哪些呢？

[This is a Sepolia **Testnet** transaction only]

Transaction Hash: 0xcbb216f23a81d86621f3c97d45a9573801d17a88f7b4742773bff7c1a93b2143

Status: Success

Block: 10106243 4359 Block Confirmations

Timestamp: 14 hrs ago (Jan-23-2026 01:26:24 PM UTC)

From: 0xC5F529D83e2eB294f20AD33ebe3a2D3BbB9aFb63

To: [0x6d88f5533203691d936d755918c2e4804a2b29e4 Created]

Value: 0 ETH

Transaction Fee: 0.000465385084855243 ETH

Gas Price: 3.617760437 Gwei (0.000000003617760437 ETH)

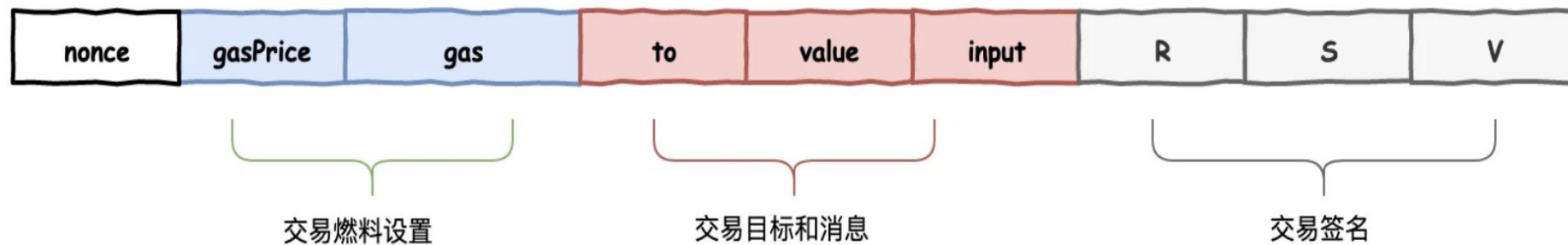
Gas Limit & Usage by Txn: 129,732 | 128,639 (99.16%)

Gas Fees: Base: 2.117760437 Gwei | Max: 4.406890091 Gwei | Max Priority: 1.5 Gwei

Burnt & Txn Savings Fees: Burnt: 0.000272426584855243 ETH (\$0.00) Txn Savings: 0.000101512849560906 ETH (\$0.00)

以太坊交易结构

下图是以太坊交易数据结构，根据用途，我将其划分为四部分。



transfer实现

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract TransferExample {
    function sendViaTransfer(address payable _to) public payable {
        // transfer会发送msg.value数量的以太币到_to地址
        // 如果失败, 会回滚交易 (抛出错误)
        _to.transfer(msg.value);
    }
}
```


send实现

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SendExample {
    event SendSuccess(address to, uint amount);
    event SendFailure(address to, uint amount);

    function sendViaSend(address payable _to) public payable {
        // send返回布尔值表示是否成功
        bool sent = _to.send(msg.value);
        if (sent) {
            emit SendSuccess(_to, msg.value);
        } else {
            emit SendFailure(_to, msg.value);
        }
    }
}
```


call实现

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract CallExample {
5     event CallSuccess(address to, uint amount);
6     event CallFailure(address to, uint amount);
7
8     function sendViaCall(address payable _to) public payable {  undefined gas
9         // call返回一个bool值表示是否成功, 和一个bytes类型的返回数据
10        (bool sent, bytes memory data) = _to.call{value: msg.value}("");
11
12        if (sent) {
13            emit CallSuccess(_to, msg.value);
14        } else {
15            emit CallFailure(_to, msg.value);
16        }
17    }
18
19    // 使用call调用另一个合约的函数
20    function callExternalContract(address _target, bytes calldata _data)  infinite gas
21        public
22        payable
23        returns (bool, bytes memory)
24    {
25        return _target.call{value: msg.value}(_data);
26    }
27 }
28
```


三者的相同点

相同点：

都可以在合约之间相互转账，send和call都有返回值。需要在代码中判断返回值。

- `addr.transfer(1 ether)`、`addr.send(1 ether)`、`addr.call.value(1 ether)`的接收方都是`addr`。
- 如果使用`addr.transfer(1 ether)`、`addr.send(1 ether)`，`addr`合约中必须增加`fallback`回退函数！
- 如果使用`addr.call.value(1 ether)`，那么被调用的方法必须添加`payable`修饰符，否则转账失败！

三者的不同点

不同点：

1. transfer

- 如果异常会转账失败，抛出异常(等价于require(send()))（合约地址转账）
- 有gas限制，最大2300
- 函数原型：<address payable>.transfer(uint256 amount)

2. send

- 如果异常会转账失败，仅会返回false，不会终止执行（合约地址转账）
- 有gas限制，最大2300
- 函数原型：<address payable>.send(uint256 amount) returns (bool)

3. call

- 如果异常会转账失败，仅会返回false，不会终止执行（调用合约的方法并转账）
- 没有gas限制
- <address>.call(bytes memory) returns (bool, bytes memory)

03

深入理解Create2

合约实操

Create2使用都有哪些场景呢？



Create2使用场景

1. 去中心化交易所的流动性池

在去中心化交易所（如 Uniswap）中，每个交易对都会有一个独立的流动性池合约，这些合约的地址由交易对的两个代币决定。如果使用 `create2` 部署流动性池合约，开发者可以确保对于给定的代币对（例如 ETH/USDT），流动性池的合约地址在创建前就可以被确定。这使得以下场景成为可能：

- **预先计算和确认流动性池地址：**用户或其他合约可以在交易对的流动性池实际部署前计算出该池的地址，从而提前与其互动或做出决策（如锁定资产、设置权限）。
- **流动性池的重建：**如果某个流动性池合约因为某种原因被销毁，使用相同的 `salt` 和初始化代码，流动性池可以在相同地址重新部署，保证系统的一致性。

Create2使用场景

2. 分布式应用的模块化设计

在某些分布式应用（DApps）中，可能会有多个模块化合约，这些合约之间通过地址直接交互。`create2` 可以确保这些模块的合约地址在开发阶段就已知，从而简化合约之间的依赖管理。例如：

- **模块间依赖**: 如果一个模块的合约需要调用另一个模块的特定合约功能，在使用 `create2` 部署合约的情况下，调用方合约可以在开发时就知道被调用合约的地址，从而避免在运行时传递不必要的参数。
- **升级机制**: 如果某个模块需要升级，只要使用相同的 `salt` 部署新版本的合约，旧的模块地址可以指向新部署的合约，从而简化了升级过程。

Create2使用场景

3. 预言机系统

在某些情况下，预言机系统需要与合约进行通信，并确保结果被发送到一个确定的地址。使用 `create2`，可以在预言机系统中提前设定目标合约的地址：

- **可信数据推送**: 预言机可以在链下计算并确认目标合约的地址，确保推送的数据到达正确的接收合约，而不会因为地址变化导致误发送。

Create2使用场景

4. 钱包和智能合约账户

在一些钱包或智能合约账户的实现中，用户的账户地址可能是基于某些参数（如用户的公钥）生成的。

`create2` 允许在部署这些账户合约之前，预先计算出这些地址：

- **账户地址的确定性**: 用户可以知道他们的智能合约钱包地址在特定参数下是固定的，即使钱包合约被销毁并重新部署，地址依然保持不变。
- **助记词恢复**: 如果智能合约钱包的地址是基于用户的助记词或其他确定性输入生成的，`create2` 允许用户通过相同的输入恢复他们的钱包。

Create2使用场景

5. 多方签名和托管合约

在多方签名（multi-signature）和托管合约（custodial contracts）的应用中，使用 `create2` 可以确保合约的地址是预先确定的，从而在合约实际部署之前就可以依赖它进行多方协调：

- **多方签名协议：**多方在协议初期就可以商定并确定未来多签合约的地址，这样可以在合约部署前进行签名、资产锁定等操作。
- **托管合约的透明性：**在资产托管系统中，托管合约的地址可以提前确定，资产可以提前转移到预定地址，而不会受到合约部署顺序的影响。

Create2工作原理

使用 CREATE2 部署的合约的地址计算如下：

```
address = keccak256(0xff ++ senderAddress ++ salt ++ keccak256(initCode))[12:]
```

其中：

- `0xff`：是常量字节，如果表示成10进制就是255。
- `senderAddress`：是创建新合约的合约地址。
- `salt`：创建者选择的32字节的值
- `initCode`：是待创建合约的初始化代码

Create2工作原理

使用 CREATE2 部署的合约的地址计算如下：

```
address = keccak256(0xff ++ senderAddress ++ salt ++ keccak256(initCode))[12:]
```

其中：

- `0xff`：是常量字节，如果表示成10进制就是255。
- `senderAddress`：是创建新合约的合约地址。
- `salt`：创建者选择的32字节的值
- `initCode`：是待创建合约的初始化代码

Create2部署和使用步骤

- 1、部署 `Factory` 合约: 部署 `Factory` 合约到以太坊网络上。
- 2、计算 `SimpleContract` 合约字节码的哈希值: 调用 `getBytecodeHash` 函数, 传递 `_value` 参数, 获取 `SimpleContract` 的字节码哈希值。
- 3、计算合约地址: 调用 `computeAddress` 函数, 传递 `salt` 和前一步获取的 `bytecodeHash`, 计算将要部署的 `SimpleContract` 合约的地址。
- 4、部署 `SimpleContract` 合约: 调用 `deploy` 函数, 传递 `salt` 和 `_value` 参数, 使用 `CREATE2` 操作码部署 `SimpleContract` 合约。

这里有一个细节需要注意: `salt`需要传递一个`bytes32`的类型, 如果你输入一个`int`类型, 需要转换一下。

通过上述步骤, 可以使用 `CREATE2` 操作码在部署之前计算合约地址, 并在部署时确保合约被部署到预期地址。



04

多签钱包演示

什么是多签钱包

多签钱包 (Multi-Signature Wallet) 是一种智能合约，要求多方（而不是单一地址）共同签名才能执行关键操作，比如转账、修改配置等。它的核心思想是“分散信任”，避免单点故障。想象一下，一个DAO的资金由一个私钥控制，如果这个私钥丢了或被盗，整个项目就GG了！多签钱包通过要求M-of-N签名（比如3-of-5，5个签名者中至少3个同意）来降低风险。

多签钱包场景和功能

在Solidity中，多签钱包通常用于：

- **团队资金管理：**比如DAO的国库需要核心成员共同批准才能支出。
- **安全保障：**防止单一管理员失误或恶意操作。
- **去中心化治理：**投票决定合约行为，比如升级或参数调整。
- **托管服务：**在交易中确保资金安全，只有多方确认后释放。

多签钱包的核心功能包括：

- **提交提案：**某个签名者提出一个交易（比如转ETH）。
- **确认提案：**其他签名者投票支持。
- **执行交易：**达到所需签名数后自动执行。
- **权限管理：**添加或移除签名者，调整签名要求。

多签钱包核心实现

2. 事件定义

- `Deposit` 事件：当以太币存入钱包时触发
- `Submit` 事件：当提交交易时触发
- `Approve` 事件：当交易被批准时触发
- `Execute` 事件：当交易被执行时触发
- `Revoke` 事件：当批准被撤销时触发

3. 状态变量

- `owners`：存储所有者地址的数组
- `isOwner`：检查某地址是否为所有者的映射
- `required`：执行交易所需的最少批准数
- `transactions`：存储所有交易的数组
- `approve`：存储每个交易被每个所有者批准情况的映射

4. 结构体定义

- `Transaction` 结构体：包含交易目标地址、发送金额、交易数据和执行状态



Q&A